
*Department of Computer Science &
Engineering Education*



Information Retrieval Techniques

NAME - RAJ

ENROLLMENT NO. - NITRB-25-BD-PG-010

COURSE - M.TECH CSE(BIG DATA ANALYTICS)

GithubLink

<https://github.com/rajsanodiya122/Information Retrieval Techniques>

Unit-3

Activity 6 : Smart Tourist Guide System for Travel Experience Discovery

Problem Statement

A tourism department is building a digital platform where travelers share short descriptions of their travel experiences such as:

- “Beautiful beach with clean water and peaceful environment”
- “Crowded market with amazing street food”
- “Historic fort with rich cultural heritage”

Problems faced:

- Tourists cannot easily search relevant travel experiences
- Experiences are not grouped (nature, food, heritage, etc.)
- Hard to explore similar places

To improve user experience, the department wants to build a Smart Tourist Guide System that can:

- Quickly search travel experiences using keywords
- Automatically group experiences into categories
- Help users discover similar travel destinations

Objective

Develop a system that:

- Preprocesses travel descriptions
- Builds an inverted index for fast retrieval
- Implements a keyword-based search system
- Groups experiences using clustering
- Analyzes how clustering improves discovery

Dataset

Example Dataset is as Follows

doc_id,description

T1,Beautiful beach with clear water and relaxing atmosphere

T2,Crowded street market with delicious local food

T3,Ancient temple with spiritual environment and history

T4,Mountain trekking adventure with scenic views

T5,Modern shopping mall with luxury brands

T6,Historic fort showcasing cultural heritage

T7,Calm lake surrounded by greenery and nature

T8,Food festival with variety of traditional dishes

Real Dataset

Students can use: Travel reviews datasets (Kaggle – TripAdvisor, tourism reviews)

Methodology

Text Preprocessing

The first step involved loading a real travel review dataset containing two attributes: **Review** and **Rating**. Since the review text was unstructured, preprocessing was required to clean the data. The preprocessing steps included:

- Converting all text into lowercase
- Tokenization of review sentences into individual words
- Removing stopwords such as *the, is, and, with*
- Removing punctuation and special symbols

This process generated clean and meaningful tokens that improved the efficiency of later tasks.

```

11: import pandas as pd
import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from collections import defaultdict
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans

nltk.download('punkt')
nltk.download('stopwords')

[nltk_data] Downloading package punkt to /home/hduser/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /home/hduser/nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

```

12: True

```

```

13: df = pd.read_csv("tripadvisor_hotel_reviews.csv")

print("\n===== Dataset =====")
print(df.head())

# Create doc_id if not present
df['doc_id'] = ['T' + str(i+1) for i in range(len(df))]

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = str(text).lower()
    tokens = word_tokenize(text)

    cleaned_tokens = []

    for word in tokens:
        if word not in stop_words and word not in string.punctuation:
            cleaned_tokens.append(word)

    return cleaned_tokens

df['processed_text'] = df['Review'].apply(preprocess)

print("\n===== Preprocessed Text =====")
print(df[['doc_id', 'processed_text']].head())

```

```
[2]: df = pd.read_csv('tripadvisor_hotel_reviews.csv')

print("\n===== Dataset =====")
print(df.head())

# Create doc_id if not present
df['doc_id'] = ['T' + str(i+1) for i in range(len(df))]

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = str(text).lower()
    tokens = word_tokenize(text)

    cleaned_tokens = []

    for word in tokens:
        if word not in stop_words and word not in string.punctuation:
            cleaned_tokens.append(word)

    return cleaned_tokens

df['processed_text'] = df['Review'].apply(preprocess)

print("\n===== Preprocessed Text =====")
print(df[['doc_id', 'processed_text']].head())
```

```
===== Dataset =====
```

	Review	Rating
0	nice hotel expensive parking got good deal sta...	4
1	ok nothing special charge diamond member hilt...	2
2	nice rooms not 4* experience hotel monaco seat...	3
3	unique, great stay, wonderful time hotel monac...	5
4	great stay great stay, went seahawk game aweso...	5

```
===== Preprocessed Text =====
```

	doc_id	processed_text
0	T1	[nice, hotel, expensive, parking, got, good, d...
1	T2	[ok, nothing, special, charge, diamond, member...
2	T3	[nice, rooms, 4, experience, hotel, monaco, se...
3	T4	[unique, great, stay, wonderful, time, hotel, ...
4	T5	[great, stay, great, stay, went, seahawk, game...

Inverted Index Implementation

After preprocessing, an inverted index was created to enable fast retrieval of travel experiences. The index stored data in the format:

****Word → List of Document IDs****

For example:

* hotel → T1, T5, T12

* food → T20, T45

This allowed the system to directly locate relevant travel reviews without scanning the entire dataset repeatedly.

```
[31]: # TASK 2: Build Inverted Index
inverted_index = defaultdict(list)

for i, row in df.iterrows():
    doc_id = row['doc_id']
    words = row['processed_text']

    for word in words:
        if doc_id not in inverted_index[word]:
            inverted_index[word].append(doc_id)

print("\n==== Sample Inverted Index =====")
count = 0
for word, docs in inverted_index.items():
    print(word, "->", docs)
    count += 1
    if count == 10:
        break

===== Sample Inverted Index =====
nice -> ['T1', 'T2', 'T3', 'T4', 'T8', 'T11', 'T12', 'T13', 'T14', 'T17', 'T19', 'T20', 'T23', 'T29', 'T35', 'T36', 'T37', 'T40', 'T42', 'T
6', 'T48', 'T56', 'T57', 'T58', 'T61', 'T65', 'T67', 'T68', 'T75', 'T79', 'T82', 'T85', 'T90', 'T93', 'T95', 'T96', 'T97', 'T100', 'T103', 'T
107', 'T108', 'T109', 'T111', 'T115', 'T116', 'T120', 'T121', 'T122', 'T124', 'T125', 'T126', 'T127', 'T131', 'T134', 'T135', 'T137', 'T14
', 'T155', 'T162', 'T163', 'T164', 'T166', 'T167', 'T168', 'T173', 'T176', 'T178', 'T179', 'T181', 'T183', 'T184', 'T187', 'T193', 'T196', 'T
197', 'T199', 'T200', 'T201', 'T205', 'T209', 'T215', 'T216', 'T223', 'T225', 'T229', 'T231', 'T235', 'T236', 'T237', 'T239', 'T242', 'T24
', 'T245', 'T246', 'T247', 'T248', 'T250', 'T252', 'T253', 'T255', 'T261', 'T265', 'T266', 'T274', 'T275', 'T276', 'T277', 'T282', 'T287', 'T
294', 'T300', 'T301', 'T304', 'T305', 'T309', 'T310', 'T314', 'T319', 'T323', 'T324', 'T325', 'T326', 'T330', 'T332', 'T334', 'T336', 'T33
', 'T339', 'T340', 'T342', 'T345', 'T346', 'T351', 'T352', 'T353', 'T355', 'T356', 'T359', 'T361', 'T364', 'T369', 'T372', 'T374', 'T375', 'T
379', 'T392', 'T394', 'T395', 'T396', 'T398', 'T402', 'T405', 'T412', 'T413', 'T414', 'T417', 'T418', 'T420', 'T426', 'T427', 'T430', 'T43
', 'T432', 'T434', 'T434', 'T440', 'T448', 'T449', 'T453', 'T456', 'T457', 'T459', 'T460', 'T463', 'T466', 'T470', 'T471', 'T472', 'T473', 'T486', '
T488', 'T492', 'T498', 'T501', 'T502', 'T506', 'T507', 'T508', 'T509', 'T511', 'T516', 'T520', 'T521', 'T526', 'T527', 'T528', 'T536', 'T54
', 'T545', 'T546', 'T548', 'T550', 'T551', 'T552', 'T553', 'T557', 'T560', 'T562', 'T563', 'T567', 'T569', 'T570', 'T572', 'T573', 'T575', '
T576', 'T577', 'T578', 'T579', 'T583', 'T584', 'T586', 'T588', 'T591', 'T592', 'T593', 'T597', 'T598', 'T599', 'T604', 'T605', 'T607', 'T61
', 'T614', 'T615', 'T617', 'T619', 'T622', 'T623', 'T626', 'T635', 'T647', 'T648', 'T649', 'T650', 'T651', 'T656', 'T657', 'T658', 'T660', 'T664', 'T660', 'T670', 'T671', 'T673', 'T674', 'T690', 'T691', 'T694', 'T695', 'T696', 'T697', 'T706', 'T708', 'T711', 'T715', 'T718', 'T71
', 'T720', 'T721', 'T722', 'T725', 'T728', 'T734', 'T735', 'T737', 'T741', 'T742', 'T743', 'T746', 'T747', 'T748', 'T750', 'T753', 'T757',
```

Search System Implementation

An AND-based keyword search system was implemented where users can search queries such as:

- * “hotel parking”
- * “great stay”
- * “food service”

The system processes each query, finds matching document IDs from the inverted index, and returns reviews containing all query terms. This provided accurate and fast search results for tourists.

```
[4]: # TASK 3: AND-Based Search System

def search_experience(query):
    query_words = query.lower().split()
    result_sets = []

    for word in query_words:
        if word in inverted_index:
            result_sets.append(set(inverted_index[word]))
        else:
            result_sets.append(set())

    if result_sets:
        final_result = set.intersection(*result_sets)
    else:
        final_result = set()

    print(f"\nSearch Query: {query}")

    if final_result:
        print("Matching Travel Experiences:")
        for doc in final_result:
            result = df[df['doc_id'] == doc]
            print(doc, ":", result['Review'].values[0])
    else:
        print("No matching travel experiences found.")

search_experience("hotel parking")
search_experience("great stay")
search_experience("food service")
```

```
Search Query: hotel parking
Matching Travel Experiences:
T1751 : good value 75/night stayed night 3-4/05, parking ok small car free, staff helpful friendly, room 3 rd floor rear quite large quiet,
able walk wharf area no problems, hotel clean mustiness previous reviewer not detectable, breakfast coffee muffins fresh fruit, good value
oney,
T6488 : run loud dirty platinum priority club member disappointed stay hotel, lobby nice room left lot desired, window room n't really open
close way seal, think did seal not heard road airport noise quite bad, hairdryer did not work yes tried reset button, room tired threadbare
dirty, stayed lot holiday inns process updating, not.the indoor pool hot tub open 24-hours dingy green, enjoyed hot tub long day traveling
ure shower afterward.parking tight not spaces open special priority club parking full.the thing say management hotel great, despite problem
manager worked make happy content,
T7470 : great value pleasant stay admired exterior hotel interior turned just charming, bedspread curtains little old-fashioned taste third
```

Clustering Implementation

To improve travel discovery, TF-IDF vectorization was applied to convert textual reviews into numerical vectors. After that, the K-Means clustering algorithm was used to group similar travel experiences.

The generated clusters were:

- * **Cluster 0:** Luxury/Hotel Stay
- * **Cluster 1:** Service/Food Experience
- * **Cluster 2:** Travel Comfort/General Reviews

These clusters helped users explore similar travel experiences automatically.

```
151 | # TASK 4: TF-IDF + KMeans Clustering
      df['clean_text'] = df['processed_text'].apply(lambda x: " ".join(x))

      vectorizer = TfidfVectorizer(max_features=500)
      X = vectorizer.fit_transform(df['clean_text'])

      kmeans = KMeans(n_clusters=3, random_state=42)
      df['cluster'] = kmeans.fit_predict(X)

      print("\n==== Clustering Results =====")
      print(df[['doc_id', 'Review', 'Rating', 'cluster']].head(20))
```

```
==== Clustering Results =====
doc_id      Review      Rating  cluster
6      T1  nice hotel expensive parking got good deal sta...      4      1
1      T2  ok nothing special charge diamond member hilt...      2      1
2      T3  nice rooms not 4* experience hotel monaco seat...      3      1
3      T4  unique, great stay, wonderful time hotel monac...      5      0
4      T5  great stay great stay, went seahawk game aweso...      5      1
5      T6  love monaco staff husband stayed hotel crazy w...      5      1
6      T7  cozy stay rainy city, husband spent 7 nights m...      5      1
7      T8  excellent staff, housekeeping quality hotel ch...      4      0
8      T9  hotel stayed hotel monaco cruise, rooms genero...      5      0
9      T10 excellent stayed hotel monaco past w/e delight...      5      0
10     T11 poor value stayed monaco seattle july, nice ho...      2      1
11     T12 nice value seattle stayed 4 nights late 2007. ...      4      1
12     T13 nice hotel good location hotel kimpton design ...      4      0
13     T14 nice hotel not nice staff hotel lovely staff q...      3      1
14     T15 great hotel night quick business trip, loved l...      4      0
15     T16 horrible customer service hotel stay february ...      1      1
16     T17 disappointed say anticipating stay hotel monac...      2      1
17     T18 fantastic stay monaco seattle hotel monaco hol...      5      0
18     T19 good choice hotel recommended sister, great lo...      5      0
19     T20 hmmm say really high hopes hotel monaco chos...      3      1
```



```
[6]: # TASK 5: Analysis

print("\n==== Cluster Analysis =====")
print("Cluster 0 -> Luxury / Hotel Stay")
print("Cluster 1 -> Service / Food Experience")
print("Cluster 2 -> Travel Comfort / General Reviews")

print("\n==== Search vs Clustering Comparison =====")
print("Search retrieves exact reviews using keywords.")
print("Clustering groups similar travel experiences automatically.")

print("\n==== Conclusion =====")
print("This Smart Tourist Guide System improves travel discovery through:")
print("1. Fast keyword search")
print("2. Similar destination discovery")
print("3. Better tourist recommendations")

===== Cluster Analysis =====
Cluster 0 -> Luxury / Hotel Stay
Cluster 1 -> Service / Food Experience
Cluster 2 -> Travel Comfort / General Reviews

===== Search vs Clustering Comparison =====
Search retrieves exact reviews using keywords.
Clustering groups similar travel experiences automatically.

===== Conclusion =====
This Smart Tourist Guide System improves travel discovery through:
1. Fast keyword search
2. Similar destination discovery
3. Better tourist recommendations
```

Analysis and Interpretation

The search system works well when users know specific keywords and need exact results. However, clustering improves recommendation capabilities by grouping similar reviews together. For example, even if a user searches for hotel experiences, clustering may suggest similar luxury stays or service-related destinations.

This combination of search and clustering significantly improves tourism discovery by saving user time and enhancing recommendation quality.

Conclusion

The Smart Tourist Guide System successfully implemented all required Information Retrieval tasks. Text preprocessing improved data quality, inverted indexing enabled fast retrieval, search provided accurate results, and clustering enhanced travel recommendations. The system can be useful for tourism websites and travel applications to improve user experience and destination discovery.